# Confidential Payroll on Solana: Architecture Diagram and Requirements

Oct 1, 2026
**Shouvik(@ShouvikMohanta), Tejas(@TejasAr25219742), Saurabh(@SaurabhSingh389)**

The proof of concept pays employees from an encrypted treasury through three Anchor instructions: pay_employee, withdraw_confidential and pause_payroll. Every requirement traces to a label on a figure.

## Actors and signers

Only the employer admin and the employee sign the three MVP use cases; the auditor and the public never sign.

| Actor | LOI category | Signs | Role in the protocol |
|---|---|---|---|
| Employer admin | Direct Actor, Administrator | UC1 pay_employee, UC3 pause_payroll | Vault authority; derives the treasury decryption keys in the browser |
| Employee | Direct Actor, Beneficiary | UC2 withdraw_confidential | Receives UC1 payments without signing them; discloses on their own terms |
| Auditor | Stakeholder | Nothing | Decrypts every amount off-chain with the mint's auditor key |
| Public observer | Stakeholder | Nothing | Sees addresses and that a transfer happened, never the amount |
| Platform team | Administrator, Stakeholder | Nothing in the MVP use cases | Holds program upgrade authority and the WUSDC mint authority roles |

## MVP use cases

Three use cases make up the proof of concept. UC means use case, so UC1 to UC3 are the IDs used in every table and figure; each is one atomic state transition and exactly one Anchor instruction handler.

| UC | Atomic state transition | Anchor instruction | Signer |
|---|---|---|---|
| UC1 | Treasury confidential balance decreases and one employee's pending confidential balance increases by the same hidden amount | pay_employee | Employer admin |

| UC | Atomic state transition | Anchor instruction | Signer |
|---|---|---|---|
| UC2 | One employee's confidential available balance decreases and their public balance increases by a chosen plaintext amount | withdraw_confidential | Employee |
| UC3 | Vault.paused changes from false to true | pause_payroll | Employer admin |

Batch payroll is not a fourth use case. The client repeats UC1 once per recipient, so every payment stays a single instruction. Setup and funding instructions are listed as supporting requirements, not use cases.

## Atomic requirements and traceability

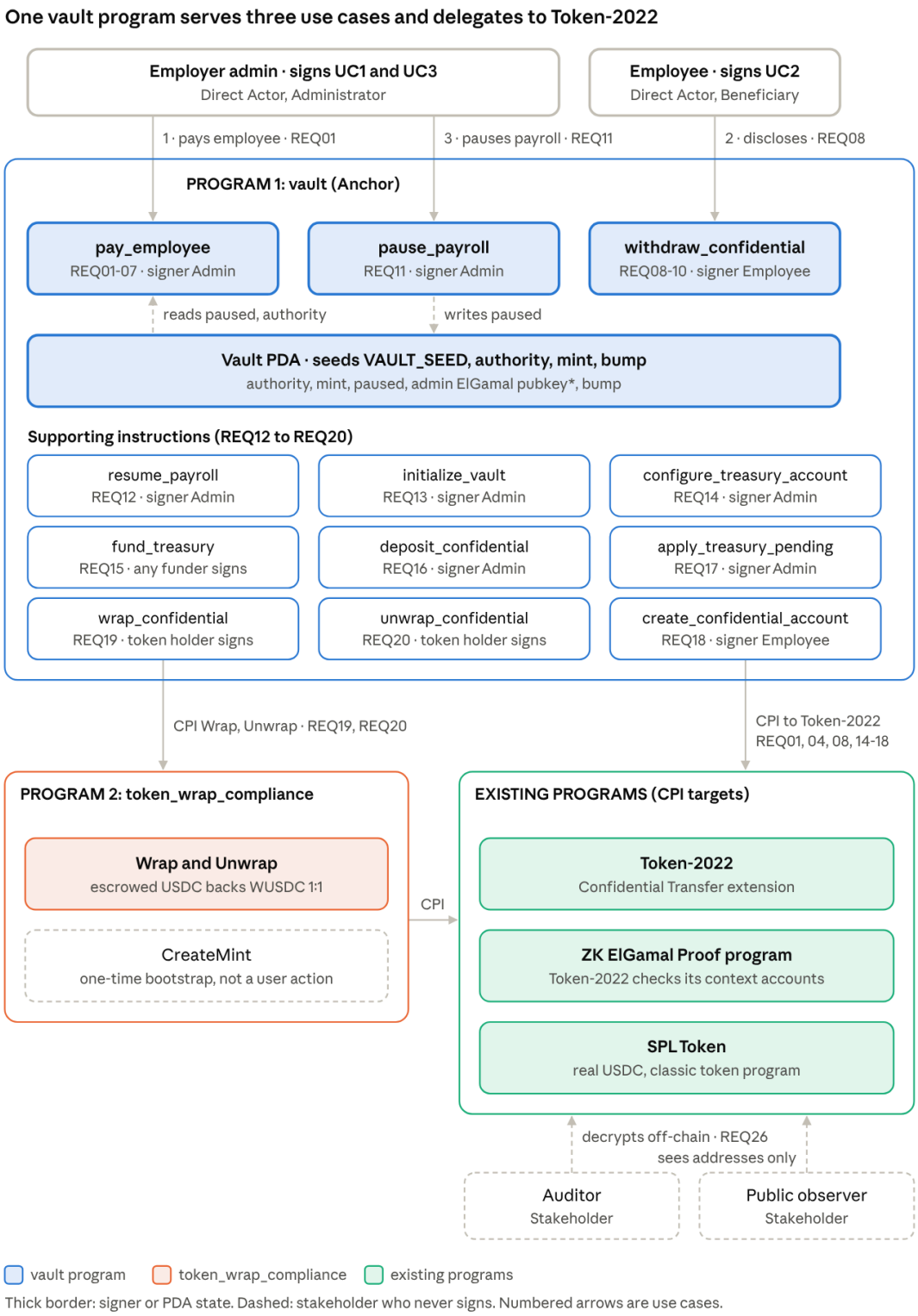
REQ01 to REQ12 cover the three use cases, REQ13 to REQ21 the supporting instructions, and REQ22 to REQ27 the client-side logic. Each row names the figure element that carries it.

| REQ | Layer | The protocol shall… | Handler | Diagram label |
|---|---|---|---|---|
| REQ01 | On-chain | transfer one hidden amount from the treasury to one employee's confidential account | pay_employee | Fig 1 arrow 1; Fig 3 box "invoke_signed Transfer" |
| REQ02 | On-chain | reject pay_employee when Vault.paused is true | pay_employee | Fig 3 "Vault.paused is false?" no branch |
| REQ03 | On-chain | reject pay_employee unless the signer equals Vault.authority | pay_employee | Fig 3 "Signer is Vault.authority?" no branch |
| REQ04 | On-chain | authorize the treasury transfer by signing as the Vault PDA | pay_employee | Fig 3 box "invoke_signed Transfer"; Fig 2 Vault PDA |
| REQ05 | On-chain | accept the transfer proofs only as three context-state account addresses | pay_employee | Fig 3 "Create 3 context accounts" |
| REQ06 | On-chain | encrypt every transfer amount to the mint's registered auditor key | pay_employee, checked by Token-2022 | Fig 3 "Build 3 proofs" and "Proofs valid and auditor key matches?" |
| REQ07 | On-chain | emit an event that carries the amount encrypted for the auditor and never the plaintext | pay_employee | Fig 3 "emit PayEmployeeEvent" |
| REQ08 | On-chain | move a chosen amount from the confidential available balance to the public balance | withdraw_confidential | Fig 1 arrow 2; Fig 4 box "invoke Withdraw" |

| REQ | Layer | The protocol shall… | Handler | Diagram label |
|---|---|---|---|---|
| REQ09 | On-chain | accept the withdraw proofs only as two context-state account addresses | withdraw_confidential | Fig 4 "Create 2 context accounts" |
| REQ10 | On-chain | emit an event that includes the plaintext amount | withdraw_confidential | Fig 4 "emit WithdrawConfidentialEvent" |
| REQ11 | On-chain | set Vault.paused to true when the authority signs | pause_payroll | Fig 1 arrow 3; Fig 5 "Set Vault.paused = true" |
| REQ12 | On-chain | set Vault.paused to false when the authority signs | resume_payroll | Fig 5 dashed box |
| REQ13 | On-chain | create the Vault PDA only for a Token-2022 mint with the ConfidentialTransferMint extension | initialize_vault | Fig 1 node; Fig 2 Vault PDA |
| REQ14 | On-chain | configure the treasury account with the Vault PDA as owner and the admin's ElGamal key as decryptor | configure_treasury_account | Fig 1 node; Fig 2 Treasury |
| REQ15 | On-chain | move WUSDC from a funder's public balance into the treasury's public balance | fund_treasury | Fig 1 node |
| REQ16 | On-chain | move the treasury's public balance into its pending confidential balance | deposit_confidential | Fig 1 node |
| REQ17 | On-chain | move the treasury's pending balance into its available balance | apply_treasury_pending | Fig 1 node |
| REQ18 | On-chain | configure an employee's own token account for confidential transfers | create_confidential_account | Fig 1 node; Fig 2 Employee account |
| REQ19 | On-chain | wrap real USDC 1:1 into WUSDC through token_wrap_compliance | wrap_confidential | Fig 1 node; Fig 6 |
| REQ20 | On-chain | burn WUSDC and release the same amount of USDC from escrow | unwrap_confidential | Fig 1 node |
| REQ21 | Roadmap | rotate the mint's auditor key | Token-2022 UpdateMint, signed by the mint's confidential-transfer authority | Fig 6 roadmap box |

| REQ | Layer | The protocol shall… | Handler | Diagram label |
|---|---|---|---|---|
| REQ22 | Client | verify treasury balance, recipient activation and unpaused vault before a run | none | Fig 3 "Simulate" |
| REQ23 | Client | create each proof context account in two transactions | none | Fig 3 and Fig 4 context boxes |
| REQ24 | Client | derive ElGamal and AES keys from a wallet signature and keep them in browser memory only | none | Fig 3 "Derive treasury keys"; Fig 4 |
| REQ25 | Client | track each recipient's status and transaction signature, and resume only unconfirmed recipients | none | Fig 6 "Off-chain database" |
| REQ26 | Client | find each recorded payment by its signature and decrypt its auditor ciphertext with the auditor's secret key | none | Fig 6 "Auditor" arrow |
| REQ27 | Client | compute split-payout parts so they sum exactly to the total | none | Fig 6 "web client" box |

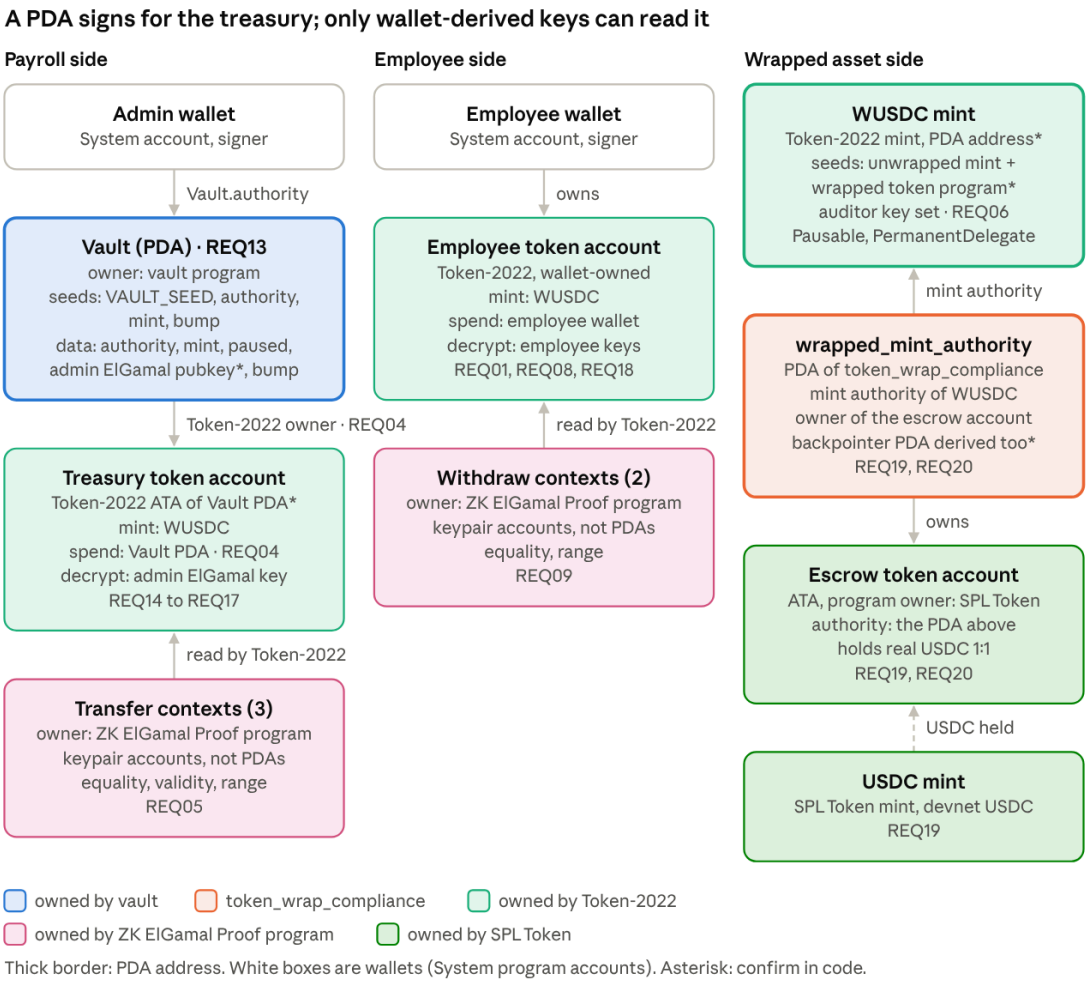
## Fig 1: Program structure




program structure · 2 programs, 12 instructions, 3 CPI targets

Three numbered arrows enter the vault program, which hands confidential-token work to Token-2022 and wrapping to token_wrap_compliance.

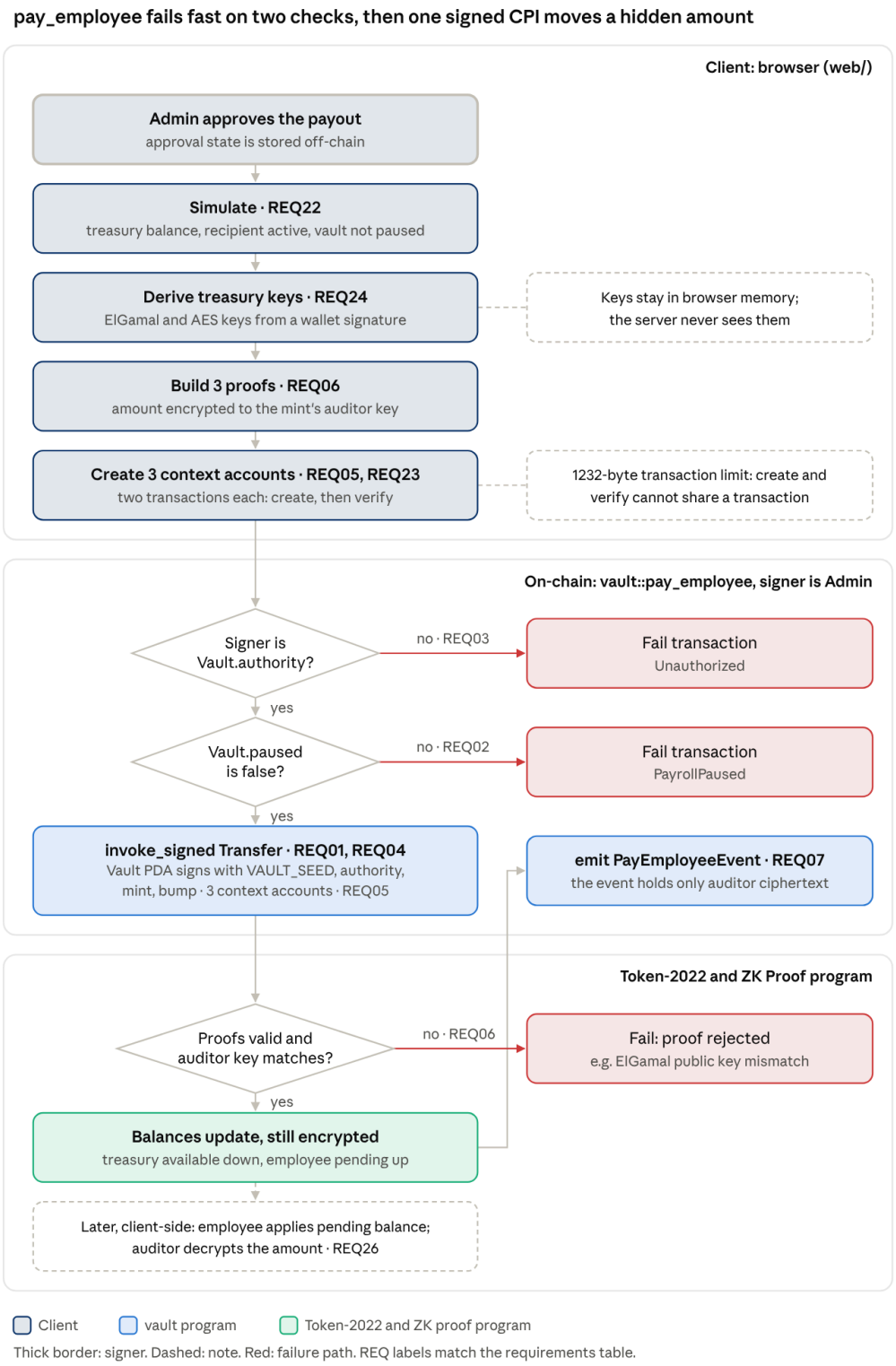
## Fig 2: Accounts and PDAs




accounts and PDAs · 3 chains, 11 boxes

Programs own every account except the wallets, and the Vault PDA and wrapped_mint_authority are the only signers that are not wallets.

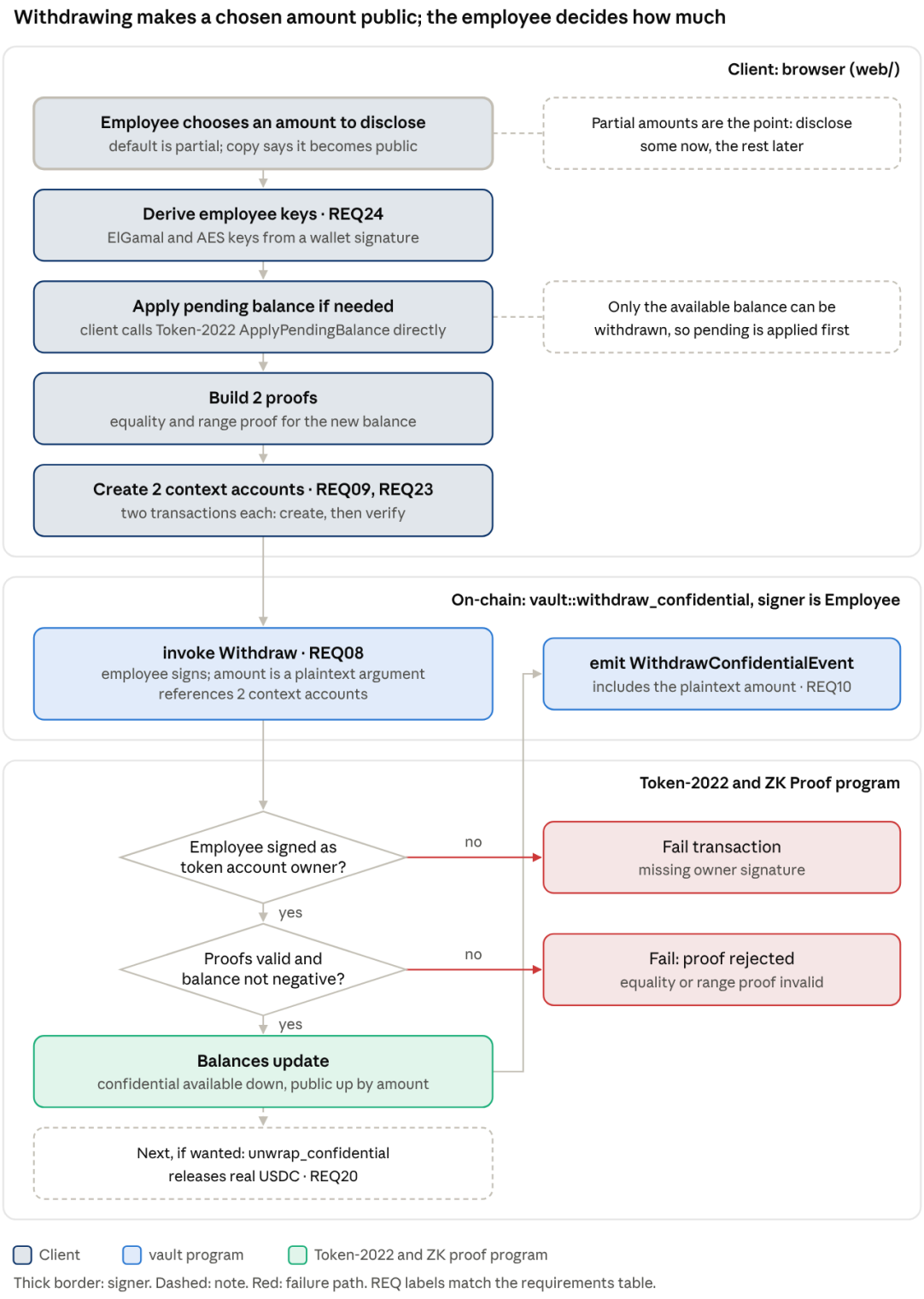
## Fig 3: Use case 1, pay_employee




pay_employee flow · 3 lanes, 3 decisions, 3 error paths

The client prepares the proofs off-chain; pay_employee checks the signer and the paused flag, then makes one signed CPI that Token-2022 verifies against the auditor key.

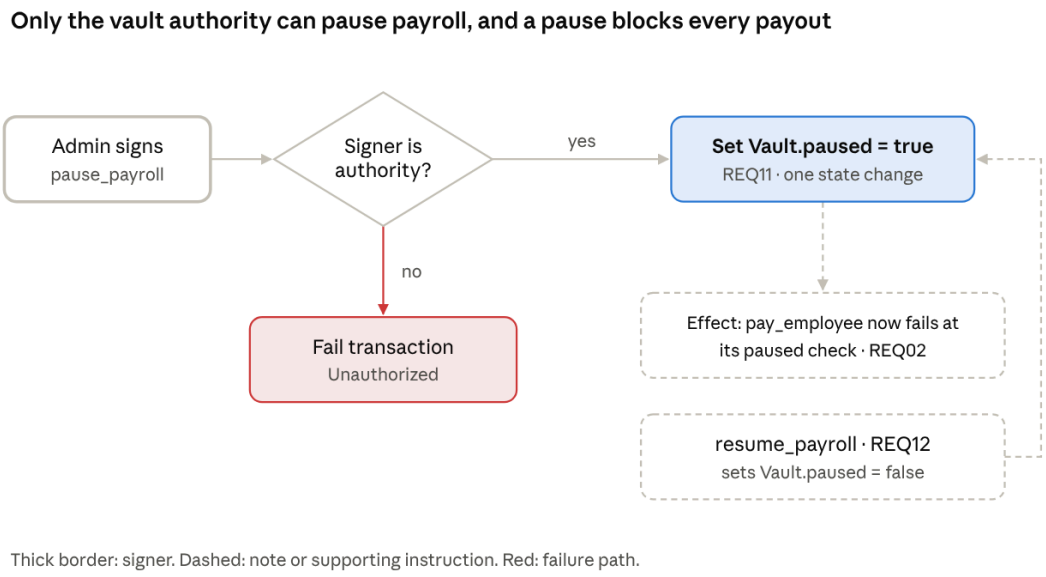
## Fig 4: Use case 2, withdraw_confidential




withdraw_confidential flow · 3 lanes, 2 decisions, 2 error paths

The amount is a plaintext argument by design: this is the one instruction where the employee chooses to make part of a balance public.

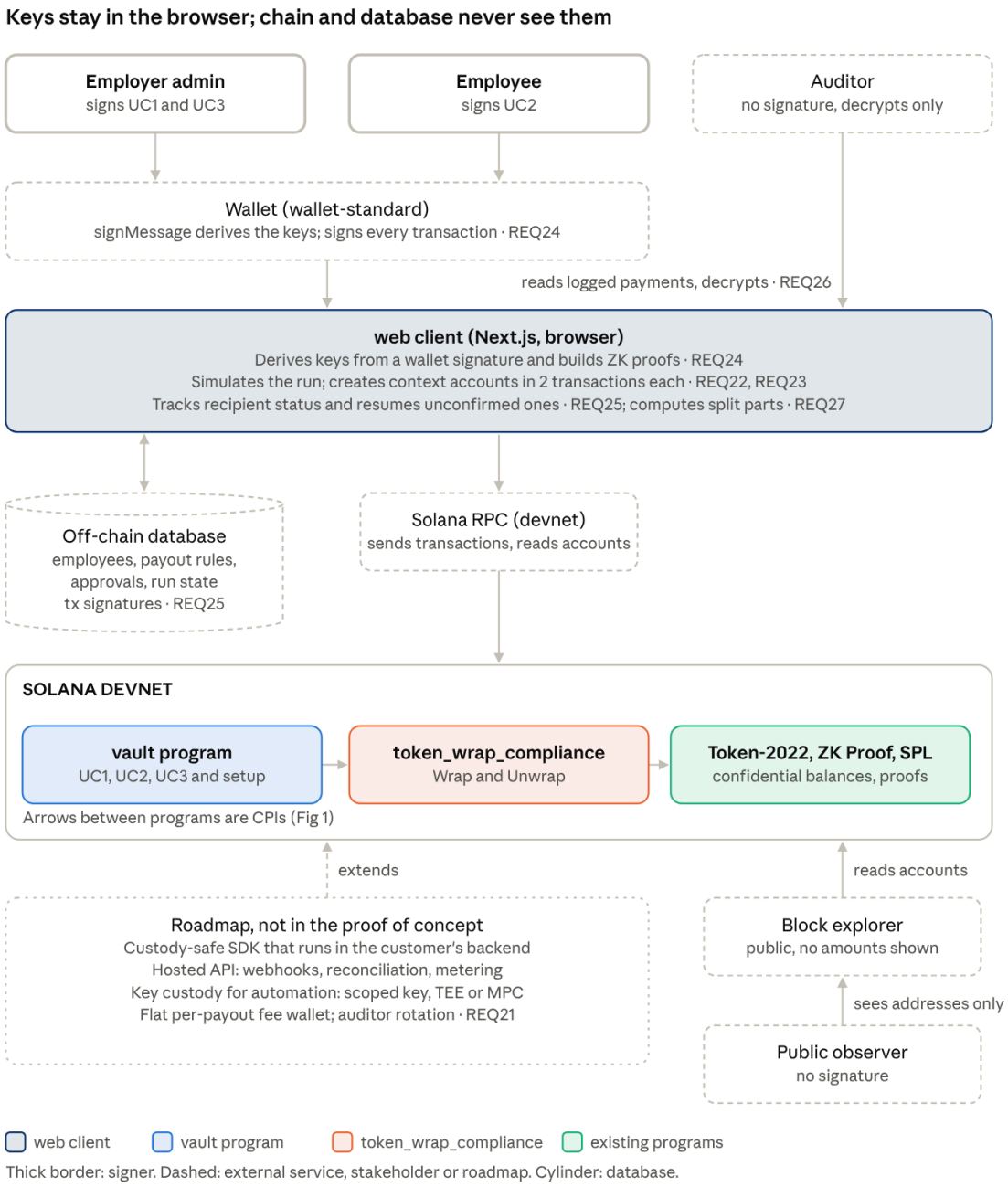
## Fig 5: Use case 3, pause_payroll




pause_payroll flow · 1 decision, 1 error path

One authority check gates the single state change, and resume_payroll reverses it under the same check.

## Fig 6: End to end with external actors




end-to-end view · 4 actors, 3 external services, 1 database, roadmap

The browser holds the keys and builds every transaction; the database stores only non-secret payroll data, and the roadmap items extend this picture without changing it.

## On-chain requirements matrix

This is the single reference list the figures trace back to. An asterisk marks a detail proposed in the design that the team should confirm while writing the code.

**Accounts**

| Account | Type | Program owner | Authority | Data and purpose | REQ |
|---|---|---|---|---|---|
| Vault | PDA | vault | Vault.authority (admin) | authority, mint, paused, admin ElGamal pubkey*, bump | REQ02, REQ03, REQ11, REQ12, REQ13 |
| Treasury token account | Token-2022 ATA of the Vault PDA* | Token-2022 | Spend: Vault PDA. Decrypt: admin ElGamal key | Confidential balance with ConfidentialTransferAccount extension | REQ04, REQ14 to REQ17 |
| Employee token account | Token-2022 account, wallet-owned (design decision) | Token-2022 | Spend: employee wallet. Decrypt: employee keys | WUSDC confidential balance | REQ01, REQ08, REQ18 |
| Transfer context accounts (3) | Keypair accounts, not PDAs | ZK ElGamal Proof program | None | Equality, ciphertext-validity and range proof results | REQ05 |
| Withdraw context accounts (2) | Keypair accounts, not PDAs | ZK ElGamal Proof program | None | Equality and range proof results | REQ09 |
| WUSDC mint | Token-2022 mint at a PDA address | Token-2022 | Mint authority: wrapped_mint_authority PDA | ConfidentialTransferMint with auditor key, PermanentDelegate, Pausable, DefaultAccountState | REQ06, REQ19, REQ20 |
| Escrow token account | ATA | SPL Token | wrapped_mint_authority PDA | Holds real USDC backing WUSDC 1:1 | REQ19, REQ20 |
| USDC mint | SPL Token mint | SPL Token | Circle | Real devnet USDC | REQ19 |

| Account | Type | Program owner | Authority | Data and purpose | REQ |
|---|---|---|---|---|---|
| Admin and employee wallets | System accounts | System program | Themselves | Sign transactions and derive ElGamal and AES keys | All |

**PDAs and seeds**

| PDA | Program | Seeds | Purpose |
|---|---|---|---|
| Vault | vault | VAULT_SEED, authority, mint, bump | Owns the treasury and signs CPIs |
| WUSDC mint | token_wrap_compliance | Derived solely from the unwrapped mint and the wrapped token program* | One canonical wrapped mint per real asset |
| wrapped_mint_authority | token_wrap_compliance | Seed prefix and inputs* | Mint authority and escrow owner |
| wrapped_mint_backpointer | token_wrap_compliance | Seed prefix and inputs* | Links the wrapped mint to the unwrapped mint |

**Instruction handlers**

| Instruction | Signer | CPIs | Constraints | REQ |
|---|---|---|---|---|
| pay_employee | Admin | Token-2022 confidential Transfer, invoke_signed | authority equals signer; paused is false; 3 context accounts | REQ01 to REQ07 |
| withdraw_confidential | Employee | Token-2022 Withdraw, invoke | 2 context accounts; amount is plaintext | REQ08 to REQ10 |
| pause_payroll | Admin | None | authority equals signer | REQ11 |
| resume_payroll | Admin | None | authority equals signer | REQ12 |
| initialize_vault | Admin | None | Mint is Token-2022 with the confidential extension | REQ13 |

| Instruction | Signer | CPIs | Constraints | REQ |
|---|---|---|---|---|
| configure_treasury_account | Admin | ATA create; Reallocate and ConfigureAccount, invoke_signed | Admin only; pubkey-validity proof at offset -1 | REQ14 |
| fund_treasury | Any funder | SPL transfer_checked | None, it only adds value | REQ15 |
| deposit_confidential | Admin* | Token-2022 Deposit, invoke_signed | Vault PDA is owner | REQ16 |
| apply_treasury_pending | Admin | Token-2022 ApplyPendingBalance, invoke_signed | Client supplies the new decryptable balance | REQ17 |
| create_confidential_account | Employee | ATA create; Reallocate and ConfigureAccount, invoke | Pubkey-validity proof at offset -1 | REQ18 |
| wrap_confidential | Token holder | token_wrap_compliance Wrap | Wrapped-mint PDAs checked with seeds::program | REQ19 |
| unwrap_confidential | Token holder | token_wrap_compliance Unwrap | Wrapped-mint PDAs checked with seeds::program | REQ20 |

**CPI dependencies**

| Caller | Callee | Calls | Signing |
|---|---|---|---|
| vault | Token-2022 | Transfer, Reallocate, ConfigureAccount, Deposit, ApplyPendingBalance | invoke_signed for the treasury, invoke for wallet-owned accounts |
| vault | Token-2022 | Withdraw | invoke, employee signs |
| vault | token_wrap_compliance | Wrap, Unwrap | invoke, token holder signs |
| vault | Associated Token Account program | Idempotent create | invoke |

| Caller | Callee | Calls | Signing |
|---|---|---|---|
| token_wrap_compliance | Token-2022 and SPL Token | Mint, burn, transfer between WUSDC and USDC | invoke_signed by wrapped_mint_authority |
| Token-2022 | ZK ElGamal Proof program | Reads context-state accounts to check proofs | Account read, no signer |

**Custom program constraints and errors**

| Constraint | Where | REQ |
|---|---|---|
| Manual TLV check that the mint has ConfidentialTransferMint | initialize_vault | REQ13 |
| Pubkey-validity proof must sit at instruction offset -1 | configure_treasury_account, create_confidential_account | REQ14, REQ18 |
| Signer must equal Vault.authority | pay_employee, pause_payroll, resume_payroll | REQ03, REQ11, REQ12 |
| Vault.paused must be false | pay_employee | REQ02 |
| Proofs referenced only as context-state accounts | pay_employee, withdraw_confidential | REQ05, REQ09 |
| Wrapped-mint PDAs validated with seeds::program | wrap_confidential, unwrap_confidential | REQ19, REQ20 |
| Auditor key must match the mint, else Token-2022 returns ElGamal public key mismatch | Token-2022, during pay_employee | REQ06 |
| Errors live in one VaultError enum; Unauthorized* and PayrollPaused* are proposed variants | error.rs | REQ02, REQ03 |

## External dependencies and milestones

The protocol has no oracle or price feed, because wrapping is 1:1 and nothing is priced. Its external dependencies are the wallet, the RPC, an off-chain database and the ZK proof stack.

| Dependency | Kind | Integration point | If it fails |
|---|---|---|---|
| Wallet (wallet-standard) | External service | signMessage derives keys; signs every transaction | Without signMessage the user cannot derive keys, so the app blocks them |

| Dependency | Kind | Integration point | If it fails |
|---|---|---|---|
| Solana RPC (devnet) | External service | Sends transactions and reads accounts | Transactions fail or slow down; the run resumes unconfirmed recipients (REQ25) |
| Off-chain database | Database | Employee list, payout rules, approvals, run state | No run can start; on-chain balances are unaffected |
| @solana/zk-sdk (WASM) | Client library | Generates proofs in the browser | A wrong key or auditor key makes Token-2022 reject the proof |
| ZK ElGamal Proof program | On-chain program | Verifies proofs into context-state accounts | An invalid proof fails its own transaction, so no transfer happens |
| Token-2022 | On-chain program | Confidential transfer, withdraw and account setup | A failed CPI reverts the whole instruction |

**Week 1:** vault setup (initialize_vault, configure_treasury_account), wrapping and treasury funding, create_confidential_account, pay_employee, pause_payroll and resume_payroll; key derivation and proof generation in the browser; one real confidential payout on devnet.

**Week 2:** withdraw_confidential and unwrap_confidential, batch pipeline with resume, employee and auditor views, split payout, demo video and submission.

## Granularity self-check

Each rule was checked against each use case by the team before the AI review.

| Rule | UC1 pay_employee | UC2 withdraw_confidential | UC3 pause_payroll |
|---|---|---|---|
| Atomicity: exactly one instruction handler? | Yes. One handler, one Token-2022 transfer CPI from the treasury to the employee's token account. | Partly. Our program has one handler, withdraw_confidential. The client first sends Token-2022's ApplyPendingBalance as a separate call. | Yes. One handler that sets Vault.paused to true. |
| State ownership: every change in a program-owned account or an explicit off-chain indexer? | Yes. The encrypted treasury balance changes in a Token-2022 account controlled by the Vault PDA; the vault program | Yes. The balances change in the employee's own Token-2022 account, | Yes. Vault.paused lives in the Vault account owned by the vault program. |

| Rule | UC1 pay_employee | UC2 withdraw_confidential | UC3 pause_payroll |
|---|---|---|---|
| | owns only the Vault account. Run history sits in the off-chain database. | controlled by the employee wallet. | |
| Real signers: explicit, with no hidden backend assumption? | Yes. The admin signs and pays fees; the Vault PDA signs the CPI through invoke_signed. Approval is stored off-chain and signs nothing. | Yes. The employee signs and pays their own fees, including the ApplyPendingBalance call. | Yes. The admin signs, and the program checks that the signer equals Vault.authority. |
| On-chain versus client-side: protocol logic separated from indexing, display and alerts? | Yes. On-chain: authority and paused checks, the signed transfer, the event. Client-side: simulation, key derivation, proofs, context accounts, run tracking. | Yes. On-chain: the withdraw with a plaintext amount, the event. Client-side: key derivation, amount choice, ApplyPendingBalance, proofs. | Yes. On-chain: the authority check and the flag change. Client-side: the Pause button. |

## AI red-team log

Findings from the AI review of the self-check and the design, with the team's decision on each.

| Finding | Caught or flagged wrong | Team decision | Reason |
|---|---|---|---|
| No data source is defined for the auditor view (rule 2) | Caught, the team had missed it | Partly accepted: app payments covered now, full indexer in Phase 2 | The auditor view reads transaction signatures from the run history and decrypts the auditor ciphertext in each event. A chain indexer that covers every payment is Phase 2 work. |
| UC1 state lives in Token-2022 accounts, with no on-chain record of a payroll run (rule 2) | Caught | Accepted as intended, extension in Phase 2 | The vault program owns only the Vault flag by design; run history stays off-chain for now. |

| Finding | Caught or flagged wrong | Team decision | Reason |
|---|---|---|---|
| Admin approval is a hidden backend assumption (rule 3) | Flagged, but not an issue in this design | Rejected for the proof of concept | There is one employer admin, who is also the signer, so the database gatekeeps no one else. |
| Fee and rent payer is not explicit (rule 3) | Caught | Accepted | The signer of each transaction pays its fees, so the employee needs their own SOL for UC2. |
| UC1 takes about seven transactions and can leave orphaned context accounts (rule 1) | Caught | Accepted, deferred to Phase 2 | Recovery and cleanup of failed runs is Phase 2 work. |
| Pause does not stop withdrawals (rule 1) | Flagged as a question | Confirmed as intended | pause_payroll blocks only pay_employee, so employees can always withdraw their own funds. |

## Assumptions to confirm during the build

1. Employee accounts are wallet-owned: the employee owns their own Token-2022 account and pays their own transaction fees.
2. The checks, error names and event names of pay_employee, pause_payroll and resume_payroll are proposals to confirm during the build.
3. Details marked with an asterisk (Vault fields, the treasury ATA, wrapped-mint seed prefixes, the deposit_confidential signer) are design proposals to fix when the code is written.
4. The WUSDC mint has one mint-level auditor key shared by every employer. The demo uses one platform-designated auditor, and its secret key must be generated and kept before the auditor view works. The view covers payments made through the app, found through the transaction signatures in the run history, and reads the auditor ciphertext that pay_employee emits in its event. The event gets that ciphertext from the ciphertext-validity context account*. A chain indexer for full coverage is Phase 2.
5. Auditor rotation (REQ21) is a direct Token-2022 UpdateMint signed by the mint's confidential-transfer authority, held by the deployer wallet unless that authority moves to the Vault PDA. It is not a vault instruction.
6. Because amounts are hidden, the program cannot enforce percentage fees or split ratios. Those are client-side policy that the auditor verifies afterwards.
7. A real cluster limits a transaction to 1232 bytes, so each proof context account is created in two transactions (REQ23).